SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

삼성청년
SW·AI아카데미

backend - servlet 2

SAMSUNG SW AI ACADEMY FOR YOUTH

목차

1. Front Controller Pattern
2. Filter
3. Listener
4. Servlet과 Multi-Thread

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

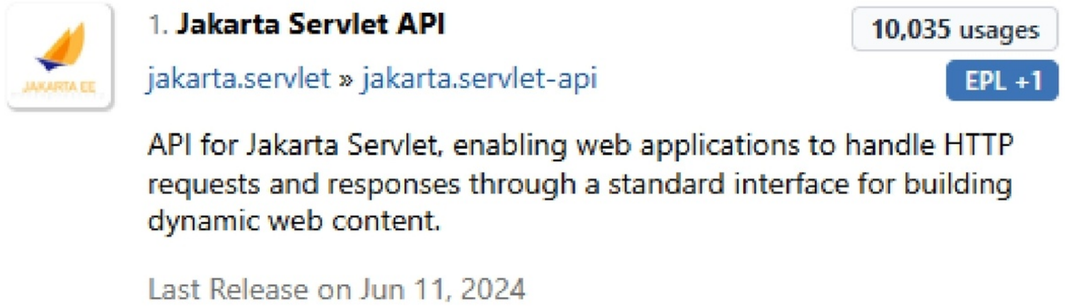
Front Controller Pattern

Maven Project 만들기 복습

Confidential

❖ 수업을 위한 Maven Project(BE_02)를 만들어보자.

- JDK 버전 확인
- 필요 의존성들



1. **Jakarta Servlet API** 10,035 usages

jakarta.servlet » jakarta.servlet-api EPL +1

API for Jakarta Servlet, enabling web applications to handle HTTP requests and responses through a standard interface for building dynamic web content.

Last Release on Jun 11, 2024



3. **Jakarta Pages API** 528 usages

jakarta.servlet.jsp » jakarta.servlet.jsp-api EPL +1

Jakarta Pages defines a templating technology for dynamically creating web resources

Last Release on Jun 11, 2024

- Maven update로 결과 확인

Front Controller Pattern

Confidential

❖ 기존 서블릿 작성 방식의 문제점

- 사용자의 요청과 이를 처리하는 Servlet이 1:1로 매칭됨
  - ◆ 요청이 추가될 때마다 매번 새로운 Servlet 생성 필요
  - ◆ 동일한 루틴으로 서블릿 생성 필요(상속, 메서드 재정의 등)



- 개별 서블릿에서
  - ◆ 비즈니스 로직과 함께 로깅, 예외 처리, 응답 처리 방식 등을 함께 처리 : 코드의 중복 발생
  - ◆ 서블릿마다 요청 처리 과정이 흩어져 있어 복잡성이 증가하고 유지 보수가 어려워짐

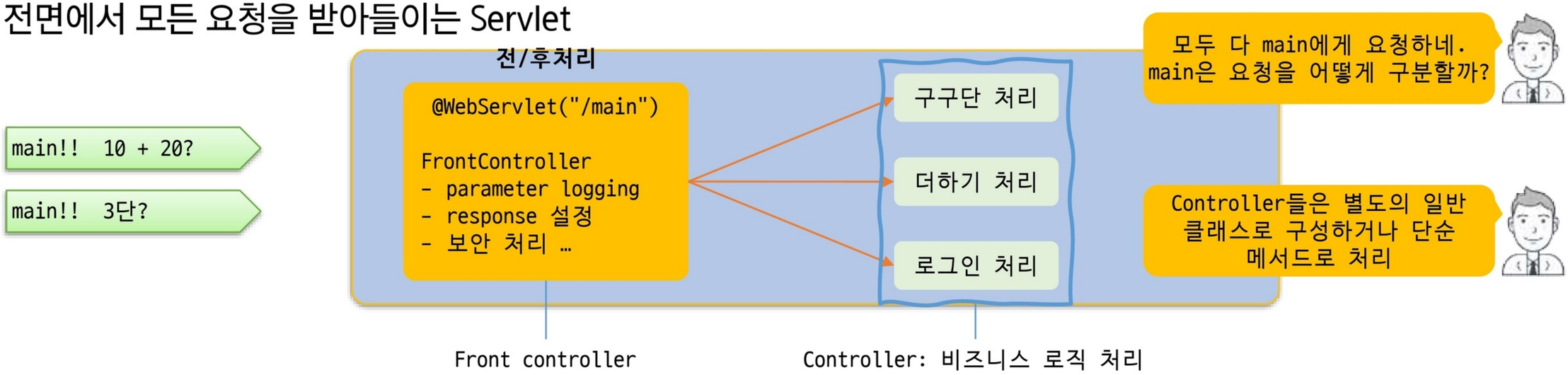
- 애플리케이션이 복잡해지면 개발생산성, 유지보수성에 문제 발생

Front Controller Pattern

Confidential

❖ Front Controller

- 전면에서 모든 요청을 받아들이는 Servlet



- front controller pattern의 장점
  - ◆ 단일 진입점: 모든 요청을 front controller에서 접수하므로 요청 처리의 일관성
  - ◆ 공통 처리: 모든 작업이 front controller를 거쳐감
    - 필요한 전/후 작업(인증, 권한 검사, 로깅 등)의 일괄 처리 가능
  - ◆ 유연한 확장성: 새로운 요청 처리를 추가할 때 기존의 구조를 크게 변경하지 않고도 쉽게 확장 가능
  - ◆ 코드 간결성: 여러 개의 Servlet을 만드는 번거로움이 줄고 코드의 가독성도 향상

Front Controller Pattern

Confidential

❖ 요청의 구분

- case 1: url에 언제나 "특정 작업"을 의미하는 파라미터(ex: action) 추가

```
/main?action=gugu&dan=3
/main?action=hello
/main?action=add&num1=10&num2=20
```

<form>에서는 action parameter를 어떻게 할까?



- case 2: 와일드 카드를 이용한 URL 매핑

URL 매핑	특성	비고
/main/* /board/*	일반적으로 업무 도메인 별로 지정된 경로 아래의 모든 요청을 받아들임 → getPathInfo() 로 호출 경로 파악 ⇒ /* 를 써버리면 정적 리소스까지 블랙홀로 빨아들이므로 주의!	/main/hello, /main/gugu?dan=3 /board/regist, /board/list
*.do	do와 같은 확장자 기반 요청을 처리하며 경로는 무관 ⇒ getServletPath()로 호출 경로 파악 ⇒ 경로 구분자 '/'를 함께 써서 경로를 한정 할 수 없음	/hello.do, /gugu.do?dan=3 /board_regist.do
/	매핑되지 않은 요청을 모두 처리한다. - 기본 web.xml의 기존의 **정적 요청에 대한 재정의 이므로 이에 대한 처리 필요** - spring mvc의 기본 mapping 방식	<servlet-mapping> <servlet-name>default</servlet-name> <url-pattern>/</url-pattern> </servlet-mapping>

아래로 갈수록 안전

Front Controller Pattern

Confidential

❖ Front Controller 작성

MainFrontController	/main	일반 요청 처리
MemberFrontController	/member	멤버 관련 요청 처리

다른 url로 시작하는 FrontController를 위해 공통 기능은 interface 활용



```
public interface ControllerHelper {
  default String getActionParameter(HttpServletRequest req, HttpServletResponse resp) {
    System.out.printf("##요청 경로: %s, 요청 방식: %s##\n", req.getRequestURI(), req.getMethod());
    req.getParameterMap().forEach((k, v) -> System.out.printf("name: %s, value: %s\n", k, Arrays.toString(v)));
    System.out.println("^^^
```

Front Controller Pattern

Confidential

❖ Front Controller 작성

```
@WebServlet("/main")
public class MainController extends HttpServlet implements ControllerHelper {
  @Override
  protected void doGet(HttpServletRequest req, HttpServletResponse resp) throws ServletException, IOException {
    String action = getActionParameter(req, resp);
    switch (action) {
      case "gugu" -> gugu(req, resp);
    }
  }
  @Override
  protected void doPost(HttpServletRequest req, HttpServletResponse resp) throws ServletException, IOException {
    String action = getActionParameter(req, resp);
    switch (action) {
      case "login" -> login(req, resp);
    }
  }
  protected void gugu(HttpServletRequest request, HttpServletResponse response) throws ServletException,IOException {
    // 구구단 처리
  }
  protected void login(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
    // 실제 login 처리
  }
}
```

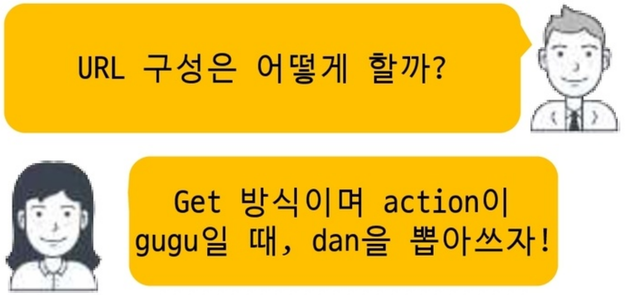
action에 따른 분기

Front Controller Pattern

Confidential

❖ 구구단 서비스해보기

- Index.html에서 front controlle를 통해서 구구단 7단을 출력해보자.
  - ◆ url 구성: `/BE_02/main?action=gugu&dan=7`



  - ◆ Servlet 구성

```
protected void doGet(HttpServletRequest request, HttpServletResponse response). . . {
  String action = getActionParameter(request, response);
  switch (action) {
    case “gugu” -> gugu(request, response);
  }
}

protected void gugu(HttpServletRequest request, HttpServletResponse response) . . .{
  int dan = Integer.parseInt(request.getParameter("dan"));
  String content = "";
  for(int i=1; i<10; i++) {
    content += String.format("%d * %d = %d<br>", dan, i, dan*i);
  }
  responseHtml("구구단("+dan+")", content, response);
}
```

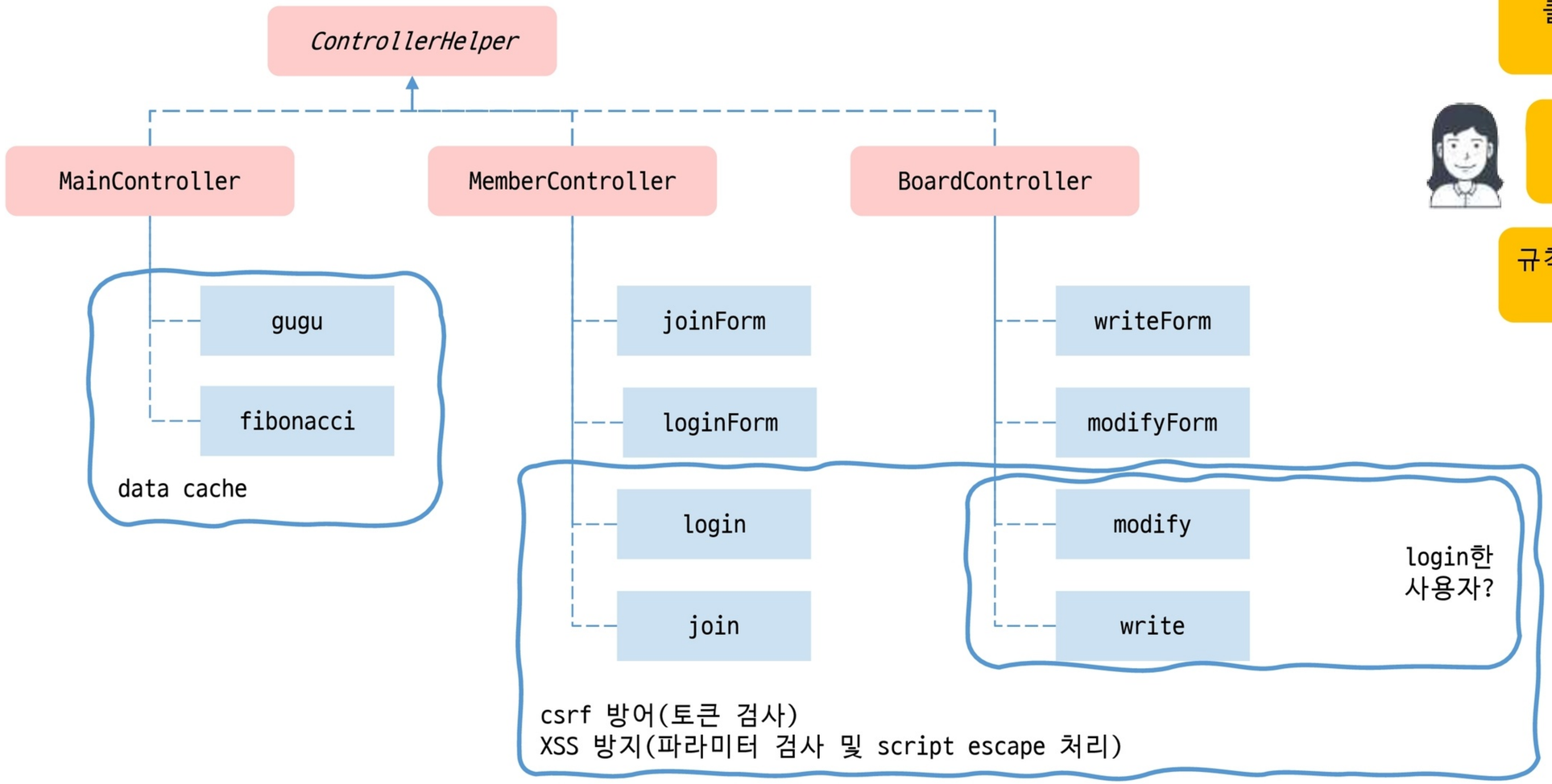
SAMSUNG SW AI ACADEMY FOR YOUTH

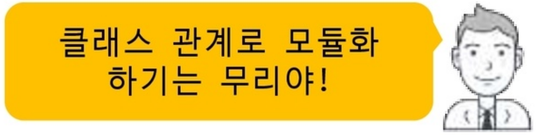
함께가요 미래로!
Enabling People

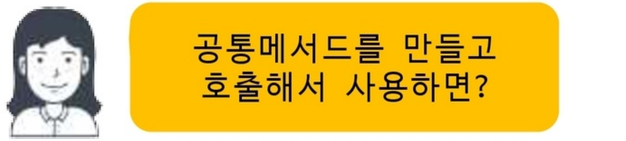
Filter

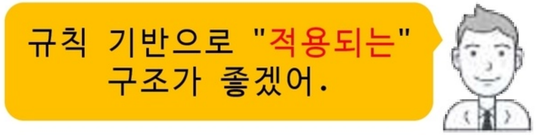
Confidential

❖ 여러 개의 컨트롤러에 부분적으로 필요한 공통 기능이 있다면?









Confidential

❖ Filter와 Filter Chain

- Filter : 웹 컴포넌트가 실행되기 전/후에 **요청과 응답을 가로채서 부가적인 기능을 수행**하기 위한 웹 컴포넌트



- Filter Chain : 여러 개의 필터가 모여 필터 체인 구성 가능

Confidential

❖ Filter의 주요 용도

- 인증 및 권한 확인 - 사용자가 특정 페이지에 접근하기 전에 로그인 했는지(인증), 권한이 있는지 확인
  - ◆ 비 로그인 또는 권한 부족 시 접근을 차단하고 로그인 페이지로 redirect 처리

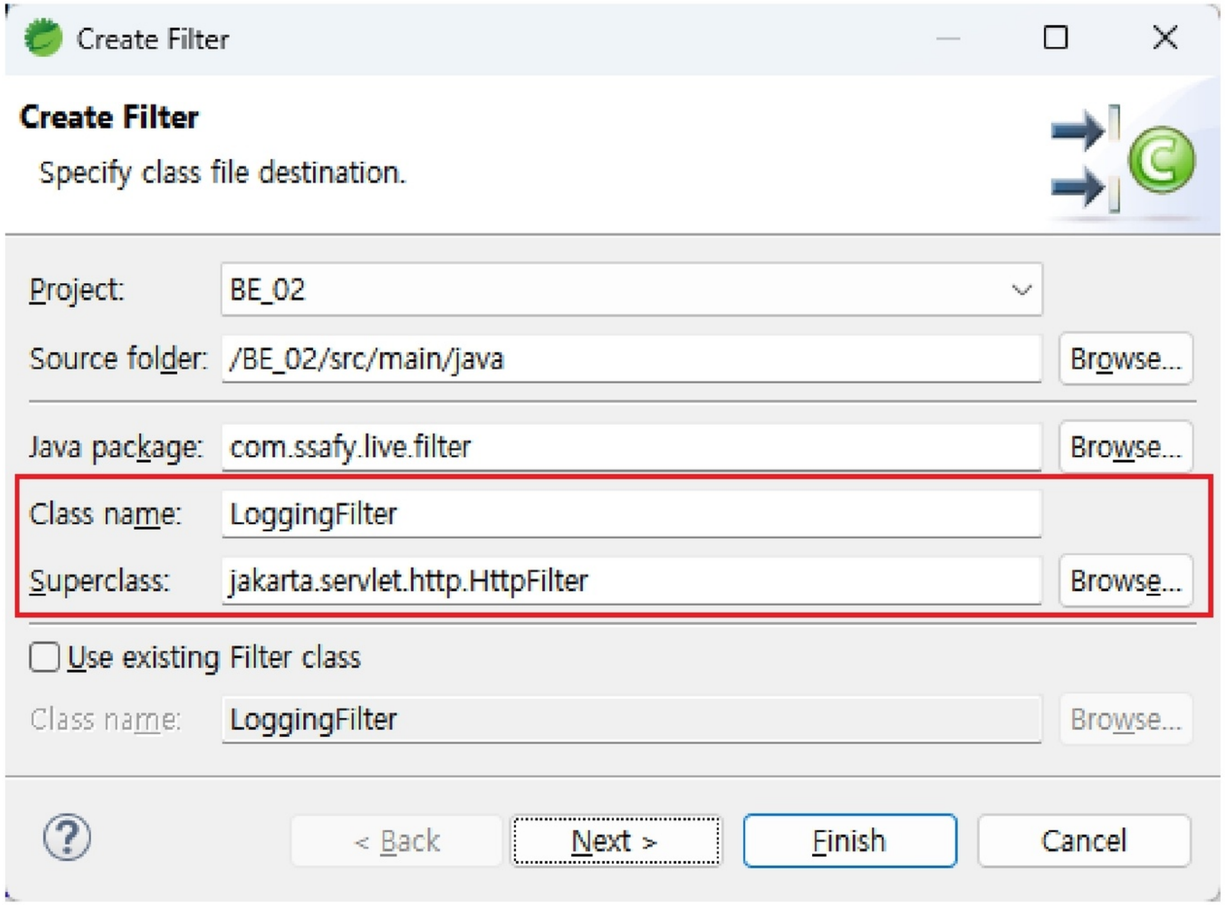
- 로깅 - 어떤 요청이 있었는지, 어떤 응답이 있었는지 추적
  - ◆ 문제 해결 및 사용자의 행동 분석

- 보안 - 요청을 검사하여 악의적인 코드나 공격 차단
  - ◆ 크로스 사이트 스크립팅(XSS) 공격 등을 방지하기 위한 입력 값 검증 등
  - ◆ < → <, > → > 등

Confidential

❖ Filter의 작성

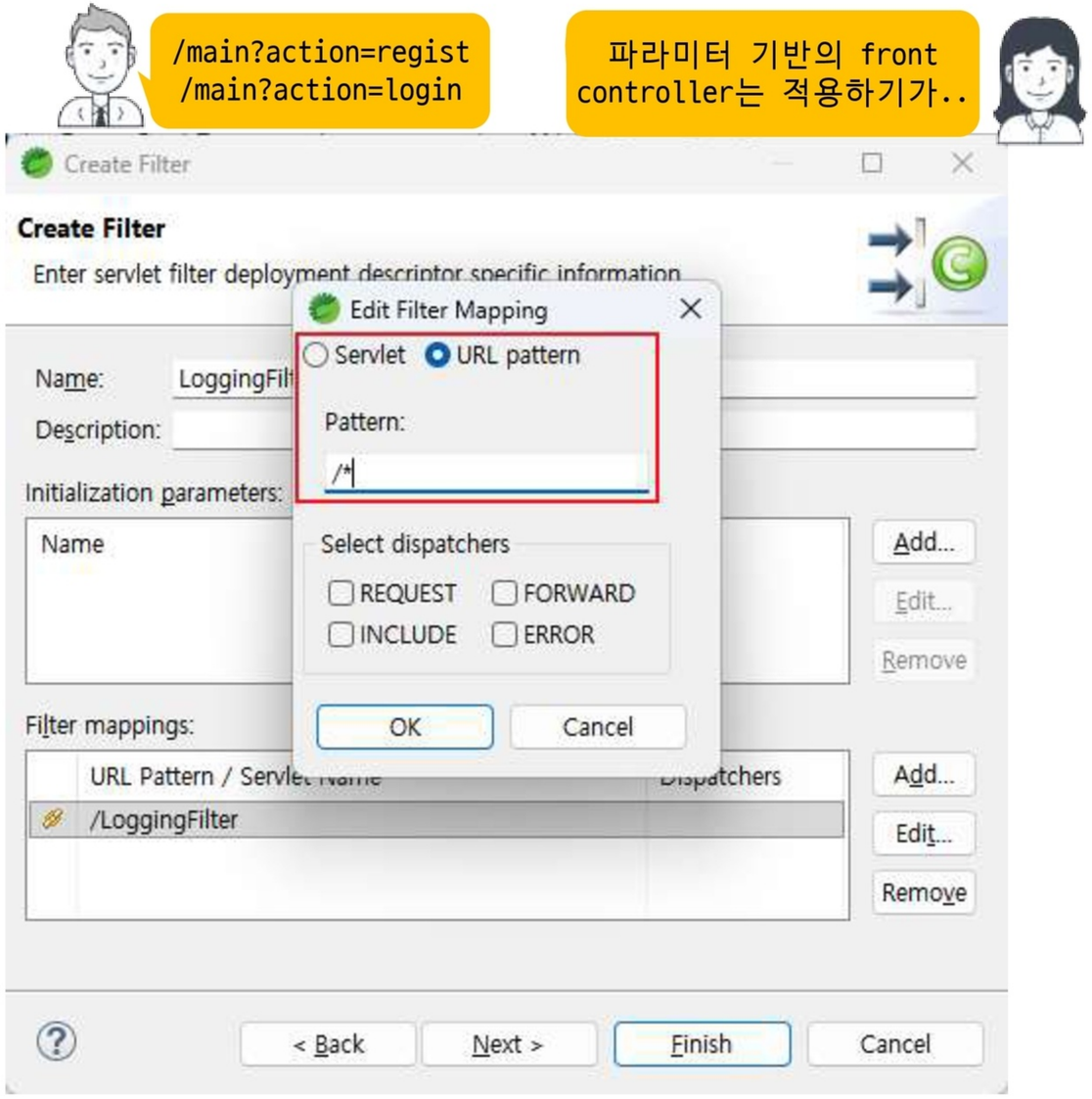
- File > new > Filter
  - ◆ jakarta.servlet.http.HttpFilter를 상속받아서 구현



Confidential

❖ Filter Mapping

- Filter 적용 대상 설정
- Servlet: filter를 적용할 개별 servlet 등록
- URL Pattern: url 기반으로 filter 적용 대상 설정
  - ◆ servlet의 url pattern과 동일
  - ◆ 경로 기반, 확장자 기반
- select dispatcher: filter를 적용할 시점
  - ◆ request : 클라이언트 직접 요청인 경우 적용(기본 값)
    - forward()는 동작 하지 않음
  - ◆ forward: forward()를 통해 이동하는 경우에 적용
  - ◆ include: include()를 통해 포함하는 경우에 적용
  - ◆ error: 에러 페이지로 전달 시
    - web.xml에 error-page 설정 필요



Confidential

❖ Filter의 작성

- @WebFilter 를 이용한 등록

```java
@WebFilter(dispatcherTypes = { DispatcherType.REQUEST }, urlPatterns = { "/*" })
public class PerformanceFilter extends HttpFilter {
  public void doFilter(HttpServletRequest request, HttpServletResponse response, FilterChain chain)
                                                                    throws IOException, ServletException {
    // do something
  }
}
```

- web.xml을 이용한 등록

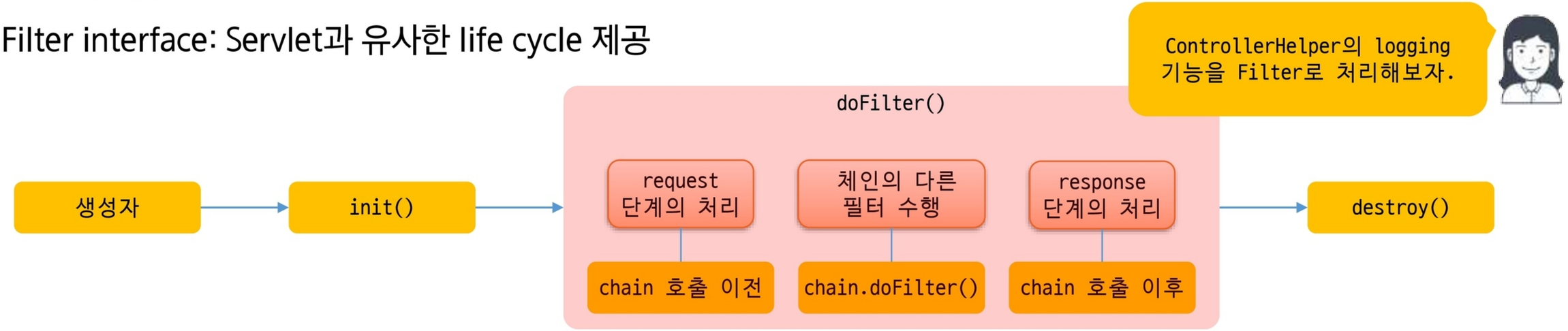
```xml
<filter>
  <filter-name>PerformanceFilter</filter-name>
  <filter-class>com.ssafy.live.filter.PerformanceFilter</filter-class>
</filter>

<filter-mapping>
  <filter-name>PerformanceFilter</filter-name>
  <url-pattern>/*</url-pattern>
  <dispatcher>REQUEST</dispatcher>
</filter-mapping>
```

Confidential

❖ Filter의 작성

- Filter interface: Servlet과 유사한 life cycle 제공




```
public void doFilter(HttpServletRequest request, HttpServletResponse response, FilterChain chain)
                                                                    throws IOException, ServletException {
    System.out.println("filter 진입");                                    HttpServletRequest 분석 및 조작

    chain.doFilter(request, response);                    filter chain의 다음 filter 호출, 없다면 Servlet 호출

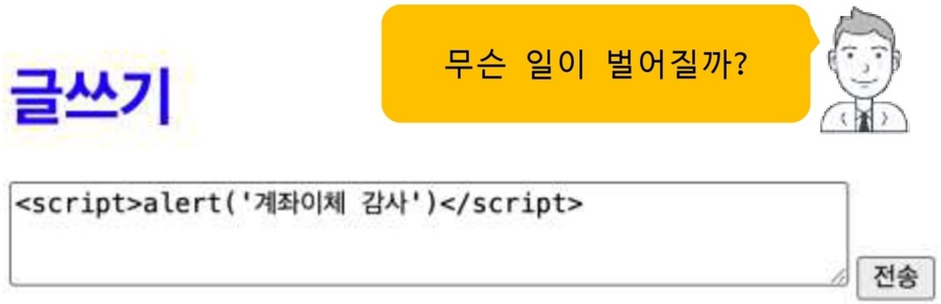
    System.out.println("응답 수신");                                    HttpServletResponse 분석 및 조작
}                                                  단, 응답은 이미 커밋된 이후 이므로 forward, redirect 호출 불가
```

Confidential

❖ XSS(Cross-Site Scripting)와 XSSFilter

- 보안 - 요청을 검사하여 악의적인 코드나 공격 차단
  - ◆ 크로스 사이트 스크립팅(XSS) 공격 등을 방지하기 위한 입력 값 검증 등
  - ◆ < → <, > → > 등으로 대체

```
<form action="/BE_02/main" method="post">
  <input type="hidden" name="action" value="write" />
  <textarea name="content" rows="3" cols="50" ></textarea>
  <button type="submit">전송</button>
</form>
```




```
@WebServlet("/main")
public class MainController extends HttpServlet implements ControllerHelper {
  . . .
  protected void write(HttpServletRequest request, HttpServletResponse response)
                                          throws ServletException, IOException {
    String content = request.getParameter("content");
    responseHtml("입력한 내용", content, response);
  }
}
```

Confidential

❖ XSS(Cross-Site Scripting)와 XSSFilter

```
@WebFilter("/*")
public class XssRequestFilter extends HttpFilter {
  public void doFilter(HttpServletRequest request, HttpServletResponse response, FilterChain chain)
                                                                    throws IOException, ServletException {
    // 요청을 Wrapper로 감싸준다.
    chain.doFilter(new XssRequestWrapper(request), response);
  }

  class XssRequestWrapper extends HttpServletRequestWrapper{
    public XssRequestWrapper(HttpServletRequest request) {
      super(request);
    }

    @Override
    public String getParameter(String name) {
      String value = super.getParameter(name);
      if(value!=null) {
        value = value.replace("<", "&lt;").replace(">", "&gt;");
      }
      return value;
    }
  }
}
```

파라미터를 직접 변경할 수 없다.



SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

Listener

Confidential

❖ Listener

- 웹 애플리케이션에서 발생하는 이벤트에 대한 모니터링 객체
- 웹 애플리케이션에서 발생하는 이벤트?

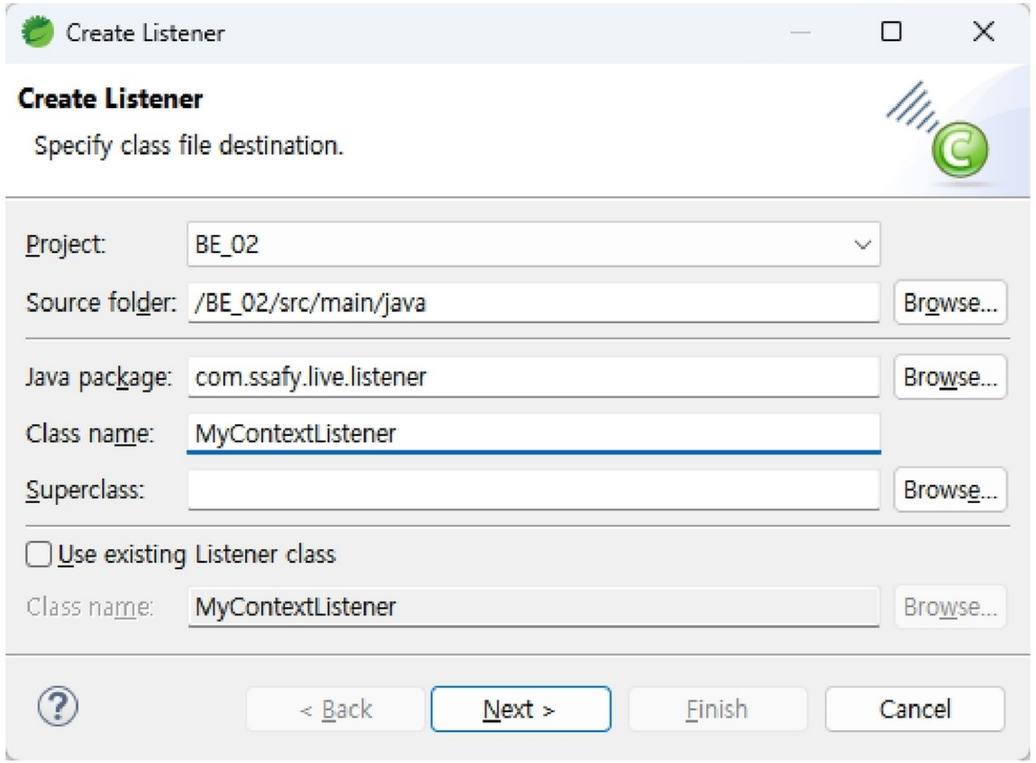


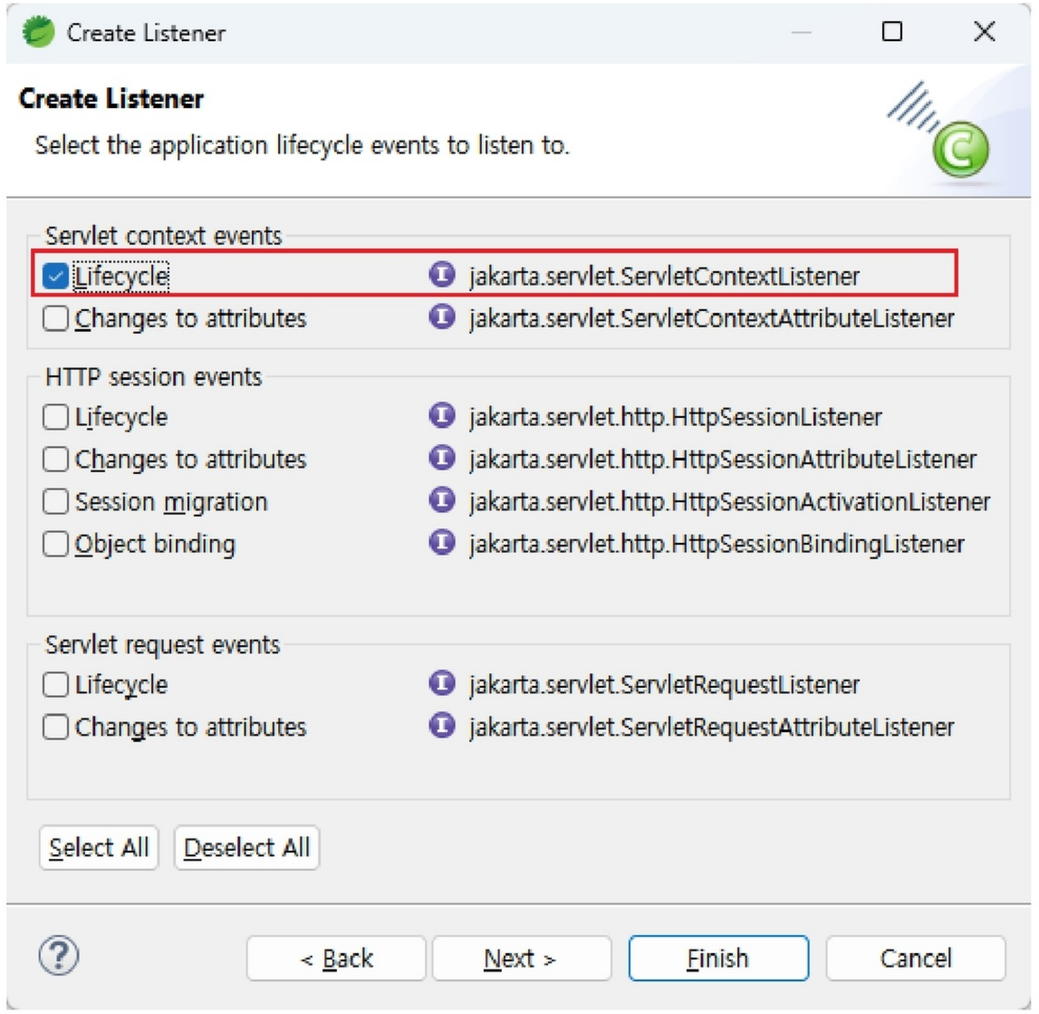
- ServletContextListener: 웹 애플리케이션 생성에서 소멸까지의 주요 사항 모니터링
  - 개별 서블릿 동작 전에 초기화하는데 비용이 많이 드는 공유 자원의 초기화에 주로 사용

Confidential

❖ Listener 생성

- File > new > Listener





Confidential

❖ Listener 등록

- @WebListener를 이용한 등록

```java
@WebListener
public class MyContextListener implements ServletContextListener {

  public void contextInitialized(ServletContextEvent sce) {
    System.out.println("context 초기화 중입니다. 전체 애플리케이션 초기화와 관련된 작업 진행");
  }

  public void contextDestroyed(ServletContextEvent sce) {
    System.out.println("context 종료합니다. 정리할 작업들 진행해주세요.");
  }
}
```

- web.xml을 이용한 등록

```xml
<listener>
  <listener-class>com.ssafy.live.listener.MyContextListener</listener-class>
</listener>
```

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

Servlet과 Multi-Thread

Servlet 특징

Confidential

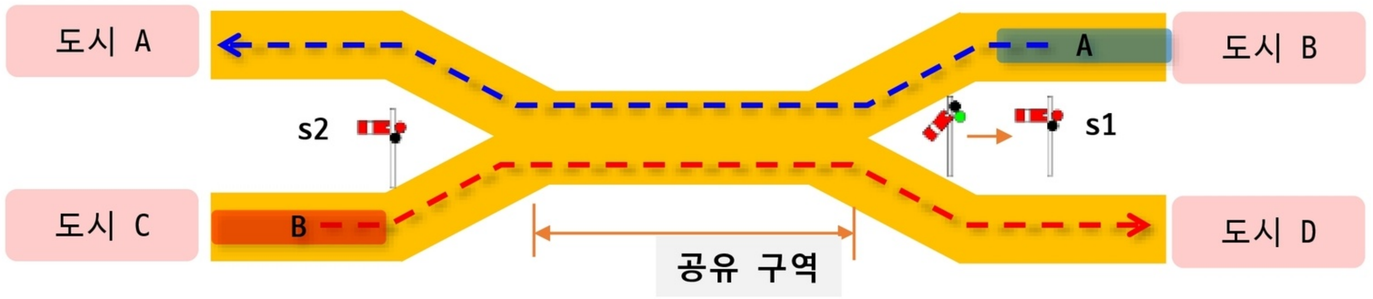
❖ Servlet의 특징 돌아보기

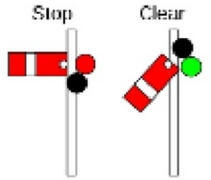
- Servlet의 장점
  - ◆ java의 OOP 기반으로 작성:
    - 유지보수성 및 재활용성 우수하며 플랫폼 독립적
  - ◆ 높은 성능:
    - 하나의 서블릿 객체만 생성하며 **멀티스레딩을 지원하여 동시 요청 처리가 가능**
  - ◆ 확장성:
    - 필터를 통한 공통 모듈의 전/후 처리, 리스너를 이용한 이벤트 기반 처리 가능 및 스프링 등 다양한 프레임워크와 통합 용이

- Servlet 단점
  - ◆ business logic과 presentation logic(HTML code)가 섞여서 나타남

Confidential

❖ multi thread와 공유 자원 문제





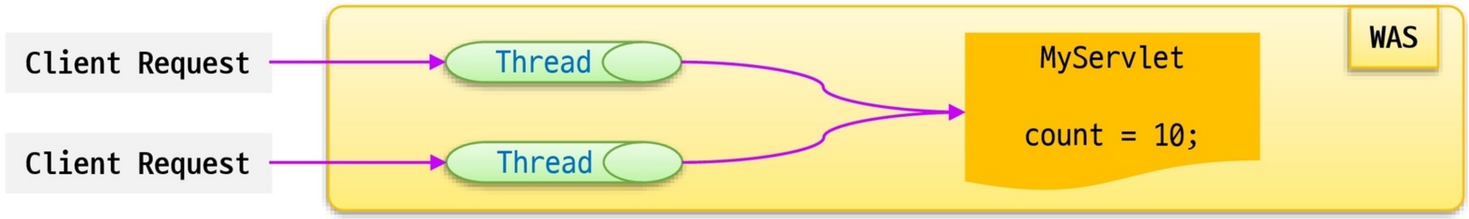
- 충돌을 방지하기 위한 방법
  - ◆ 뮤텍스(Mutex): 공유 구역에 들어갈 수 있는 통로가 단 하나
    - 자리가 하나뿐인 화장실과 열쇠 하나
    - 예: synchronized 블록
  - ◆ 세마포어(Semaphore): 공유 구역에 들어갈 수 있는 통로가 여러 개
    - 자리가 여러 개인 유료 주차장과 주차권
    - 예: connection pool

Servlet 특징

Confidential

❖ Servlet과 상태 관리

- Servlet은 Multi Thread에 의해 공유됨



  - ◆ Servlet은 공유 자원으로 Thread Safe 하지 않음
  - ◆ Servlet이 상태 정보를 가지면 데이터 훼손 문제가 발생할 수 있음

- 데이터 보호를 위해 synchronized 키워드 사용 → 한 번에 하나씩
  - ◆ synchronized를 통해 동기화 시키면 제대로 성능을 발휘할 수 없음

- 가급적 Servlet에는 멤버 변수를 통해 상태를 관리하지 않는다.

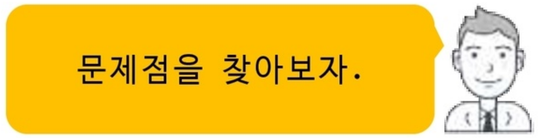
Confidential

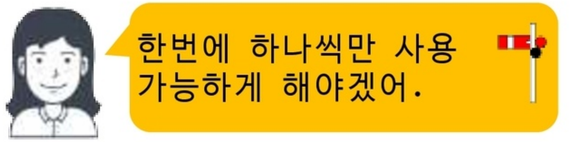
❖ 공유자원의 상태 관리

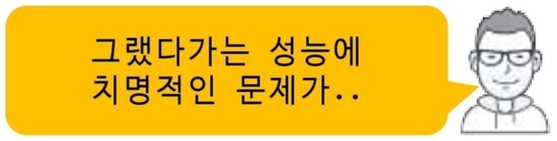
```
@WebServlet("/main")
public class MainController extends HttpServlet implements ControllerHelper {

  private long stock = 1;

  protected void buy(HttpServletRequest request, HttpServletResponse response)
                                                    throws ServletException, IOException {
    if (stock > 0) {
      try {
        Thread.sleep(1000 * 10);                    // 판매 처리 중
      } catch (InterruptedException e) {
        e.printStackTrace();
      }
      stock--;
      responseHtml("판매 성공", "이제 남은 제고는 "+stock, response);
    }else {
      responseHtml("판매 부족", "다 팔렸어요.", response);
      stock = 1;
    }
  }
}
```







내일 수업을 위해서..

Confidential

❖ 실습 파일 설치

- 수업 git에서 BE_03_lab.zip 가져오기
- 사용할 workspace에 압축 풀기(여기에 압축 풀기)



- STS : [File] - [Open Projects from File System]
  - ◆ index.jsp 동작 확인

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

내일 방송에서 만나요!

삼성청년SW·AI아카데미